

DevOps exercise 1 report

Kalle Karttunen

This is the required report for the first exercise of Tuni's DevOps course. This document will contain the system diagram, as well as reflections on some of the technical aspects of the project.

DevOps exercise 1 report	1
1. System diagram	3
2. About the development platform	3
3. About the status records	4
4. About the persistent storage solutions	4
Approach 1: A simple volume	4
Approach 2: A separate container	5
5. Difficulties and problems	5
6. How to remove the persistent data	6

1. System diagram

Below is a system diagram containing each component of the system, as well as the interactions between components.

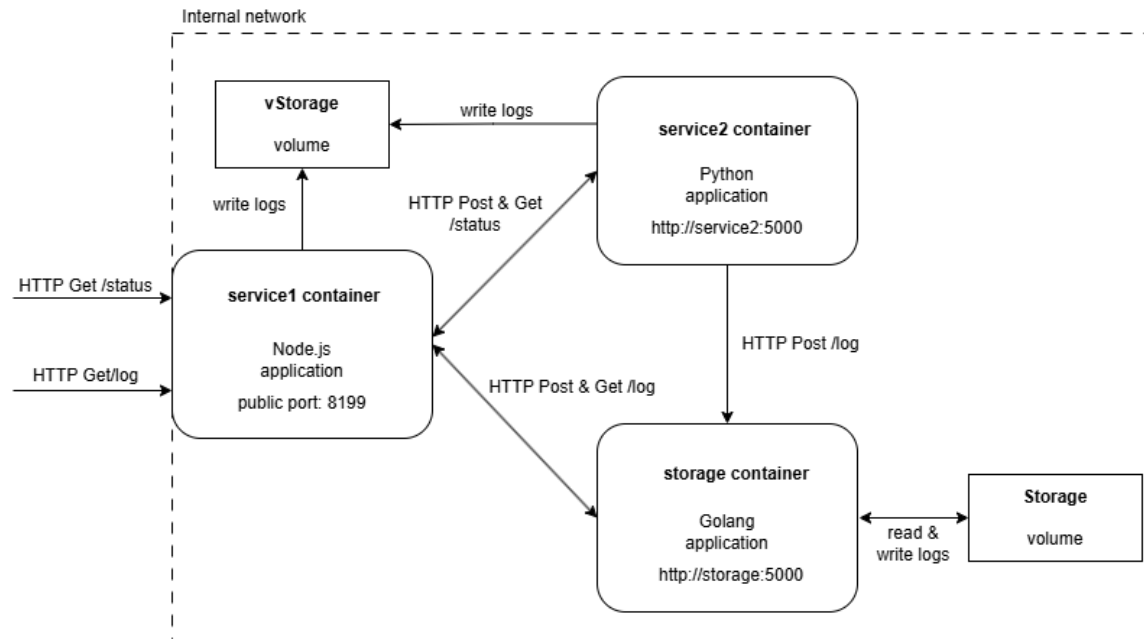


Image 1. A diagram showing each container along with their domain name within the system. It also shows the volumes, as well as the HTTP communications that each container sends and receives.

The assignment asked us to include the IP address of each container, but I did not assign IPs to any of the containers, and running `docker inspect` did not show an IP address for any container, so I included the container's DNS addresses instead.

2. About the development platform

For this assignment, I decided to just use my Windows laptop for development instead of a Linux virtual machine. Below is some basic information about my development system:

Operating system: Windows 11 Home (version 24H2)

Docker version:

Client: 28.3.3

Server: Docker Desktop 4.45.0 (203075)

Docker compose version: v2.39.2-desktop.1

3. About the status records

Both service1 and service2 analyze their status and create records about their free disk space and uptime. They both check how much free disk space is available in their root directories. However, there is a slight difference in their implementations: service2 only checks how much disk space is available for non-root users, whereas service1 checks the total available space for all users. I did this because service1 is running as the root user in its system, while service2 is not. The uptime measurement works the same for both services: it tracks the uptime of the kernel by reading from `/proc/uptime`.

As for the usefulness of these measurements, in my opinion the disk space measurement seems potentially useful, while the time measurement seems quite useless. This is because the disk capacity measurement is container-specific, which means it can actually be helpful for managing individual containers. The time measurement on the other hand is based on the host kernel, which is shared by all containers, so it can't produce any information about specific containers in a system like this. All that measurement shows me is how long I've had Docker Desktop open.

Next time, the time measurement could be improved and made container-specific by starting some type of timer the moment the application starts, instead of reading from `/proc/uptime`. Also, the scale of both measurements could be adjusted. Recording the available space in megabytes resulted in big numbers that were not that easy to read. It could be implemented in such a way that for example if the capacity exceeds 1000 MB, then gigabytes are shown instead. Also, for the time measurement, it could be more accurate than just hours. Minutes and even seconds could be added.

4. About the persistent storage solutions

For this system, I implemented two alternative storage solutions: The first one was a simple volume that could be directly accessed and written to by service1 and service2. The second one was a separate container that used its own volume which no other container had access to, and it communicated with service1 and service2 through HTTP messages. Below are my pros and cons for each approach:

Approach 1: A simple volume

Pros:

- Extremely simple, does not add much complexity into the system

- Very fast and easy to implement: just add a few lines into the docker-compose file.

Cons:

- Very unsafe: Allowing multiple services to write into the same volume can lead to issues. If one service messes up the volume, then all the services that rely on it could be compromised.
- Debugging could become more difficult if numerous services are handling the same resource

Approach 2: A separate service

Pros:

- Adds safety and stability. Reducing the number of actors that can directly modify the storage should reduce the risk it being broken or misused in some way. It's much easier to make sure one service manages the storage properly than it's to make sure that every single service that uses it does it properly.
- Isolating the storage into its own service makes it possible for it to be developed and maintained independently.

Cons:

- Adds complexity into the system, making it more challenging to develop and maintain.
- Adding another service consumes more resources, such as network and computing capacity

5. Difficulties and problems

I would say that I had surprisingly few difficulties with this assignment, considering how rusty I was with Docker. Maybe the biggest challenge was knowing where to start. I did not have any idea which file I should start writing first and what kind of implementation I should use for any of the services, and I had to do a decent amount studying before I got the ball rolling.

One issue I struggled with was getting service1 to write into the vStorage volume. The container just kept crashing when I tried to test the /status command, and I had no idea what to do. Then I remembered that there are also logs inside the containers, and those revealed that it was a file permissions issue. After trying a couple of different fixes, I just

ended up running the whole container as the root user. I'm not very happy with this fix, as it doesn't seem like the safest option, but it was the only way I could get it working.

Another issue I ran into was the massive build time of the storage container. After adding it, the build time of the whole system jumped up from something like under 20 seconds to well over a minute. This was very annoying while I was still writing the code and having to regularly rebuild the containers to test my changes. Apparently, this was caused by the huge number of packages that need to be installed when running the standard Golang image. I managed to find a fix for this on the internet: by using `debian:bookworm-slim` as the base image in the dockerfile, the image became much faster to build.

6. How to remove the persistent data

All the persistent data of the system is stored in two basic named Docker volumes, so removing it is simple. Just run these following commands:

```
docker compose down
```

```
docker volume rm devops_Storage
```

```
docker volume rm devops_vStorage
```